

FinSync: Smart Expense Tracker — Comprehensive Project & Technical Report

FinSync is a robust, production-ready, full-stack personal finance management application designed to give users granular control over their budgeting, manual expenses, and bank transaction feeds. The platform incorporates advanced user security protocols (OTP-based double-factor email confirmation, alphanumeric password validations), real-time interactive widgets, SVG-based client-side visualization, and secure CSV/Email data export engines.

1. Project Folder Directory Structure

Below is the directory map of the FinSync repository, outlining the modular distribution of responsibilities across the frontend and backend:

```
FS Project/
├── backend/
│   ├── src/
│   │   ├── config/
│   │   │   └── db.js                # MongoDB connection & auto-migration script
│   │   ├── controllers/
│   │   │   ├── auth.controller.js  # Register, login, OTP verify, change/reset passwords
│   │   │   ├── bank.controller.js  # Simulated OAuth linkage & bank transaction import
│   │   │   └── expense.controller.js # Expense CRUD, CSV compilation, email report
│   │   ├── generation
│   │   │   ├── middleware/
│   │   │   │   └── auth.middleware.js # JWT payload decryption & route guard middleware
│   │   │   ├── models/
│   │   │   │   ├── expense.model.js # Expense transaction Mongoose schema
│   │   │   │   └── user.model.js     # User profile, bank metadata, OTP Mongoose schema
│   │   │   ├── routes/
│   │   │   │   ├── auth.routes.js   # /api/auth routing definitions
│   │   │   │   ├── bank.routes.js   # /api/bank routing definitions
│   │   │   │   └── expense.routes.js # /api/expenses routing definitions
│   │   │   ├── utils/
│   │   │   │   └── email.js          # Nodemailer transactional SMTP config & HTML
│   │   ├── templates
│   │   │   ├── mock_otp.js          # Automated local e2e testing OTP generator utility
│   │   │   └── index.js              # Main server listener & middleware configuration
│   │   ├── .env                     # Backend environment configuration
│   │   ├── package.json              # Node dependencies & start scripts
│   │   └── package-lock.json
│   └── frontend/                    # React Client Application
│       ├── public/                  # Static public assets
│       ├── src/
│       │   ├── assets/
│       │   │   └── raw_logo.jpg      # Branding logo asset
│       │   ├── components/
│       │   │   ├── BankSync.jsx      # Interactive banking sync widget & OAuth form
│       │   │   └── Dashboard.jsx     # Financial dashboard coordinator & SVG metrics
```

			└ ExpenseTable.jsx	# Filtered transaction ledger, CSV & Email buttons
			└ App.css	# Custom global visual animations
			└ App.jsx	# React root entry, routes coordinator, landing page
			└ index.css	# CSS variables, dark/light definitions, glassmorphism
			└ main.jsx	# React Virtual DOM mounting point
			└ eslint.config.js	# Code linting parameters
			└ index.html	# Single-page HTML shell
			└ package.json	# Frontend dependencies & Vite dev-server config
			└ package-lock.json	
			└ vite.config.js	# Vite build config
			└ Project_Report.md	# Technical report
			└ README.md	# Project documentation
			└ package.json	# Workspace dependencies manager

2. System Architecture & Component Communication

FinSync utilizes a decoupled architecture where the React frontend client makes asynchronous HTTP requests (using the browser's Fetch API) to the Node/Express backend REST API, authenticated via JSON Web Tokens (JWT) embedded in HTTP headers.

```
sequenceDiagram
    autonumber
    actor User
    participant Client as React Client (App / Dashboard)
    participant Server as Express Server (index.js)
    participant Auth as Auth Middleware (JWT Verify)
    participant Database as MongoDB (Mongoose Models)
    participant Email as Nodemailer (SMTP Service)

    User->>Client: Input Credentials & Register
    Client->>Server: POST /api/auth/register
    Server->>Database: Save Unverified User & OTP
    Server->>Email: Send Verification OTP email
    Email-->>User: Delivers 6-Digit Code
    User->>Client: Submits OTP
    Client->>Server: POST /api/auth/verify-otp
    Server->>Database: Toggle user isVerified to true
    Server-->>Client: Returns JWT Token + User Data

    Note over Client,Server: Authenticated Actions (Headers: Authorization: Bearer <Token>)
    User->>Client: Requests Bank Integration (e.g., Kotak Mahindra)
    Client->>Server: POST /api/bank/sync (Token)
    Server->>Auth: Validate JWT Sign & Expiry
    Auth->>Database: Fetch User Document
    Server->>Database: Insert 6-10 Mock Transactions for user
    Server->>Database: Update user.linkedBank settings
    Server-->>Client: Success message & transaction ledger
    Client-->>User: Renders dynamic SVG charts & expense table
```

3. Tech Stack Deep Dive

Frontend Application

- **Core Runtime & Build Tool:** React 18, Vite. Vite enables instantaneous hot-module replacement (HMR) and optimized build bundling.
- **Styling Architecture:** Built using a custom Vanilla CSS library (`index.css` and `App.css`) mapping visual parameters via CSS Custom Properties (`--bg-primary` , `--accent-primary` , etc.) to provide toggleable **Dark/Light Themes** with smooth transitions:

```
:root {
  --bg-primary: #0b0f19;
  --glass-bg: rgba(19, 26, 48, 0.6);
  --accent-primary: #6366f1; /* Indigo */
  --text-primary: #f8fafc;
}
.light-mode {
  --bg-primary: #f8fafc;
  --glass-bg: rgba(255, 255, 255, 0.7);
  --accent-primary: #4f46e5;
  --text-primary: #0f172a;
}
```

- **SVG Analytics Engine:** Instead of incorporating heavy third-party plotting frameworks, charts are calculated manually inside `Dashboard.jsx` using standard SVG elements. This minimizes bundle sizes and preserves pixel-perfect CSS animation capabilities.

Backend Application

- **Routing & Controllers:** Express (v5) is configured with `express.json()` payload parsing and global CORS rules (`cors` package) permitting local dev-environments.
- **Database Layer:** MongoDB acts as a document store. Mongoose provides schema validation, indexes (e.g., email uniqueness), and query aggregation.
- **Security Utilities:** Passwords are saved hashed via `bcrypt` (10 rounds). Tokens are issued using standard HMAC-SHA256 signatures (`jsonwebtoken`) with an expiration period of 30 days.
- **SMTP Delivery:** Nodemailer integrates directly with SMTP host servers (supporting TLS/SSL tunnels) to mail raw transactional notifications.

4. Database Models Code Definition

A. User Schema Model (`backend/src/models/user.model.js`)

This schema manages profile credentials, bank synchronization metadata, and OTP lifetimes.

```
const mongoose = require('mongoose');
const bcrypt = require('bcrypt');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
```

```

    required: [true, 'Name is required'],
    trim: true,
  },
  email: {
    type: String,
    required: [true, 'Email is required'],
    unique: true,
    trim: true,
    lowercase: true,
    match: [/^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/, 'Please add a valid email address'],
  },
  password: {
    type: String,
    required: [true, 'Password is required'],
    minlength: [8, 'Password must be at least 8 characters long'],
  },
  linkedBank: {
    provider: { type: String, default: null },
    lastSync: { type: Date, default: null },
    connected: { type: Boolean, default: false },
  },
  monthlyIncome: { type: Number, default: 5500 },
  isVerified: { type: Boolean, default: false },
  otp: { type: String, default: null },
  otpExpires: { type: Date, default: null },
  resetPasswordOtp: { type: String, default: null },
  resetPasswordOtpExpires: { type: Date, default: null },
},
{ timestamps: true }
);

// Hash password before saving to database
userSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
});

// Compare password inputs
userSchema.methods.comparePassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);

```

B. Expense Schema Model (backend/src/models/expense.model.js)

This schema documents single transaction records. It has a relational field reference `userId` that maps back to the corresponding `User` document.

```
const mongoose = require('mongoose');

const expenseSchema = new mongoose.Schema(
  {
    userId: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: [true, 'User ID is required'],
    },
    title: {
      type: String,
      required: [true, 'Expense title is required'],
      trim: true,
    },
    amount: {
      type: Number,
      required: [true, 'Amount is required'],
      min: [0.01, 'Amount must be greater than zero'],
    },
    category: {
      type: String,
      required: [true, 'Category is required'],
      enum: {
        values: ['Food', 'Utilities', 'Entertainment', 'Transport', 'Shopping',
'Healthcare', 'Housing', 'Other'],
        message: '{VALUE} is not a supported category',
      },
      default: 'Other',
    },
    date: {
      type: Date,
      required: [true, 'Date is required'],
      default: Date.now,
    },
    source: {
      type: String,
      required: [true, 'Source is required'],
      default: 'Manual',
    },
  },
  { timestamps: true }
);

module.exports = mongoose.model('Expense', expenseSchema);
```

5. Key System Implementation Details

A. Simulated OAuth Bank Synchronizer

The client-side `BankSync.jsx` simulates authenticating credentials against a banking server. Once the user submits their credentials, the client starts a 4-second animation that cycles through progress percentages and updates status messages (e.g., *Establishing secure tunnel...*, *Parsing statement ledgers...*).

Once the simulation hits 100%, it calls the backend POST endpoint `/api/bank/sync`. The controller selects a random number of transactions (between 6 and 10) from `MOCK_MERCHANTS`, assigns them a random timestamp from the last 30 days, generates a random amount within category-specific bounds, and saves them to MongoDB under the user's ID:

```
// Sample configuration mapping category-specific bounds
const MOCK_MERCHANTS = [
  { title: 'Starbucks Coffee', minAmount: 4.5, maxAmount: 12.0, category: 'Food' },
  { title: 'Uber Trip', minAmount: 12.5, maxAmount: 45.0, category: 'Transport' },
  { title: 'Amazon Online Retail', minAmount: 15.0, maxAmount: 250.0, category: 'Shopping' },
],
{ title: 'Metropolitan Housing Rent', minAmount: 1200.0, maxAmount: 1200.0, category:
'Housing' }
];
```

B. Mongoose Analytical Aggregate Report Pipeline

To compute category shares for the doughnut chart, the controller groups and aggregates transactions dynamically:

```
const report = await Expense.aggregate([
  {
    $match: {
      userId: req.user._id,
      date: { $gte: start, $lte: end },
    },
  },
  {
    $group: {
      _id: '$category',
      totalAmount: { $sum: '$amount' },
      count: { $sum: 1 },
    },
  },
  { $sort: { totalAmount: -1 } },
]);
```

C. Client-Side SVG Doughnut Math

In `Dashboard.jsx`, categories are mapped to an SVG circular path. To segment each category, the component calculates the `strokeDasharray` and `strokeDashoffset` dynamically:

```
const radius = 50;
const circumference = 2 * Math.PI * radius; // ~314.159

// During render map:
const percent = (cat.amount / totalSpent) * 100;
const strokeLength = (percent / 100) * circumference;
```

```
const strokeOffset = circumference - (accumulatedPercentage / 100) * circumference;
accumulatedPercentage += percent;
```

6. Email Report & NodeMailer Layout Details

When the user clicks the "Email List" button, the backend generates a dark-themed HTML table report. This email contains detailed transaction records and includes a CSV attachment formatted with UTF-8 BOM encoding (`\uFEFF`) to ensure support for Indian Rupees and international character structures when opened in spreadsheet software like Microsoft Excel or Google Sheets.

NodeMailer Connection Guard Configuration:

```
const transporter = nodemailer.createTransport({
  host: process.env.EMAIL_HOST,
  port: parseInt(process.env.EMAIL_PORT),
  secure: port === 465,
  auth: { user: process.env.EMAIL_USER, pass: process.env.EMAIL_PASS },
  tls: { rejectUnauthorized: false } // Bypasses self-signed local certificate constraints
});
```

7. Troubleshooting & Diagnostics

MongoDB connection failure

The database initiator (`backend/src/config/db.js`) includes safety checks. If the environment variable `MONGODB_URI` is missing or the database is offline, it prints a warning to the console and attempts to link to local port `27017` without crashing the application. This ensures diagnostic features remain operational.

Auto-migration system

To ensure compatibility with older user models, the connection script scans the collection for legacy user accounts that lack the verification flag. It automatically verifies existing accounts to prevent login issues during updates:

```
const unmigratedCount = await User.countDocuments({ isVerified: { $exists: false } });
if (unmigratedCount > 0) {
  await User.updateMany({ isVerified: { $exists: false } }, { $set: { isVerified: true } });
}
```